

BUILD YOUR OWN CLAUDE CODE PLUGIN



Five simple steps, about 15 minutes. The failure that already happened never gets a second run.

⚠ Before you start: Claude Code installed and authenticated. That is the only prerequisite.

THE GOAL	A plugin that fires on its own when your typed words match its trigger, or when you call it by name.
THE ONE RULE	Don't start from what you could build. Start from what already went wrong in your workflow.
THE CHECKS	Claude types the files. Done = you ran its green CHECK. Where a file disagrees, the page wins.

1

NAME THE PAIN · ~5 min

Write two sentences.

1. **The pain.** What went wrong in your workflow. Once is reason enough. One failure only; two joined by an “and”? Keep the worse half. Example: “*A revenue model change broke prod.*”

2. **The workflow it happens in.** The exact words your team types. Example: “*Use when editing dbt models or schema.yml files.*” Typed words fire it; vague ones (“data quality”) never do.

CHECK

· The pain sentence has no “and.”

· The workflow sentence starts with “Use when.”

· It names words your team actually types.

2

CREATE IT · PICK ONE COMMAND · ~1 min

Open a terminal. Any folder works: this puts the plugin in Claude's own home, so it loads in every session. **dbt-guard is an example name. Type your own, everywhere it appears on this page.**

OPTION 1 · it advises · almost every plugin

\$ c Claude plugin init dbt-guard

OPTION 2 · it BLOCKS an action · hook files, needs Node.js

\$ c Claude plugin init dbt-guard --with hooks

CHECK · it printed the path

~/ . Claude / skills / dbt-guard /
└─ . Claude-plugin / plugin.json
└─ SKILL.md your skill
└─ hooks/ option 2 only

Windows: ~ is C:\Users\<you>

· Root SKILL.md is correct: plugin.json says "skills":
["./"].

· Several skills? skills/<name>/ folders, one
SKILL.md each. plugin.json stays as is.

3

HAVE CLAUDE FILL IT IN · ~5 min

\$ cd ~/ . Claude / skills / dbt-guard

\$ c Claude

> Fill this plugin in. The workflow: [your workflow sentence]. The incident: [your pain sentence]

option 2? add: also write the hook that BLOCKS [the action]

What it writes:

your workflow sentence becomes the description : (that is the trigger), your pain becomes gotcha #1 in the GOTCHAS list. Option 2: also the hooks/ files; check against THE CONTRACT.

Keep it small:

a skill covers almost everything. Add an agent only for a read → trace → judge job.

CHECK · read what it wrote

Open SKILL.md yourself:

· description: uses your team's words.

· Gotcha #1 is your incident.

· Every other gotcha is true.

You are the reviewer. This is the judgment Claude cannot supply.

4

WATCH IT WORK · ~2 min

First plant a private fact: one detail in gotcha #1 Claude could never guess, like your dashboard's name. Then /exit, open a fresh session, and ask for the risky thing:

\$ c Claude

> I'm about to edit the revenue dbt model

Two ways it fires:

your words match the description : (the real test), or you type the plugin or skill name.

CHECK

· A Skill (dbt-guard) line appears: it loaded.

· The answer repeats your private fact.

· Generic advice = FAIL. Sharpen the description, /reload-plugins, re-ask.

· By hand: /dbt-guard, or /dbt-guard:< name> for a skills/ subfolder.

5

PROVE THE BLOCK · option 2 builds only · ~2 min

In the same session: attempt the forbidden action, then a harmless one. Testing by hand instead? Take the key your hook reads from THE CONTRACT below, never from your own guess (recorder line there for everything else).

CHECK · both directions · you are done

· The forbidden action does not run, and your reason comes back. Claude may reword it.

· The harmless one proceeds.

A hook you have not watched block is not a guard.

THE CONTRACT · ONLY IF YOU PICKED OPTION 2

```
hooks/hooks.json = { "hooks": { "PreToolUse": [{ "matcher": "^(Bash)$",
"hooks": [{ "type": "command", "timeout": 10,
"command": "node ${CLAUDE_PLUGIN_ROOT}/hooks/<your-script>.js" }] } } }
stdin (read it: fs.readFileSync(0, 'utf8')) → {tool_name, tool_input, cwd...}
Edit/Write/MultiEdit → tool_input.file_path · NotebookEdit → ..notebook_path
Bash → tool_input.command · any other key (file text incl.): record it:
append the raw stdin string to a file; act once, read it, DELETE the line
exit 0 allow · exit 2 BLOCK, stderr = teach · other exits allow · anchor ^(<...>)$
```

FAIL-OPEN by default:

a crash, a wrong script path, or a timeout (seconds) each let the action run; try/catch + exit 2 hardens crashes only (a timed-out hook is killed from outside). \${CLAUDE_PLUGIN_ROOT} = this plugin's folder; node --check after edits. A hook sees the tool call, not your repo: a seatbelt, not a security boundary.

WHEN IT BREAKS · SILENT FAILURES

skill never fires

· vague description (rewrite with typed words), or frontmatter has disable-model-invocation: true; delete that line.

hook never fires

· hooks.json shape wrong (copy it from the contract), no node prefix, or stale session → /reload-plugins; /hooks must list it.

hook allows what it should block

· it never read that tool's key, or a crash let the action through; re-run step 5 per tool.

after sharing: install not found

· the marketplace names disagree with the plugin name; diff the files. validate never compares them.